

Projet : Kotlin P00

Partie 1 : Les bases de la P00 et l'encapsulation

I. La classe Arme

1. Implémentation de la classe

Créer une classe Arme avec :

- Deux propriétés :

- **nom** : le nom de l'arme (String – non modifiable)
- **degat** : les dégâts de l'arme (Int – modifiable)

Vous devez implémenter un *setter personnalisé* pour cet attribut afin de :

- ▶ vérifier que la nouvelle valeur est strictement positive ;
- ▶ mettre à jour la valeur uniquement si la condition est respectée et afficher le message suivant:
« **Modification des dégâts de l'arme NomArme.** »
- ▶ si la condition n'est pas respectée afficher le message suivant: « **Valeur invalide pour les dégâts** »

- Une méthode :

- **toString()** pour que l'affichage d'une arme soit sous la forme :
(NomArme, Degat)

2. Tests de la classe

- Créer une arme Épée avec 15 dégâts puis afficher ses caractéristiques avec la méthode toString().

Résultat attendu :

(Épée, 15)

- Créer une arme Arc avec 10 dégâts puis afficher ses caractéristiques avec la méthode toString().

Résultat attendu :

(Arc, 10)

- Modifier les dégâts de l'Arc à 12 puis afficher ses caractéristiques avec la méthode toString().

Résultat attendu :

Modification des dégâts de l'arme Arc.

(Arc, 12)

- Modifier les dégâts de l'Arc à -5

Résultat attendu :

Valeur invalide pour les dégâts

II. La classe Joueur

1. Implémentation de la classe

Créer une classe Joueur avec :

- Quatre propriétés :

- **nom** : le nom du joueur (String – non modifiable)
- **vie** : points de vie du joueur (Int – modifiable)
- **armeEquipee** : l'arme actuellement équipée (Arme – modifiable).
- **inventaire** : une liste d'armes que le joueur possède (MutableList<Arme>)

- Deux constructeurs :

- **constructeur primaire** : permet de créer un objet Joueur avec une arme équipée et l'arme : **Arme(Poing, 5)** est présente par défaut dans l'inventaire.
- **constructeur secondaire** : permet de créer un joueur sans arme équipée. Dans ce cas, l'arme **Arme(Poing, 5)** est l'arme équipée par défaut et l'inventaire est vide.

L'arme équipée n'est pas forcément présente dans l'inventaire.

- Six méthodes :

- **attaquer()** retourne les dégâts de l'arme équipée.
- **recevoirDegats(degat: Int)** diminue les points de vie du joueur avec les dégâts passé en paramètre. Attention, les points de vie ne peuvent pas être négatifs (minimum 0).
- **ajouterArme(arme:Arme)** ajoute une arme dans l'inventaire.

- **changerArme(nomArme: String)** équipe une arme depuis l'inventaire et remet l'arme précédente dans l'inventaire. (aide : voir les méthodes `.add()`, `.remove()` et `.find()` sur les listes en Kotlin)
- **afficherInventaire()** affiche toutes les armes de l'inventaire
- **toString()** affichage d'un joueur soit sous la forme :
(Nom, vie=Vie, arme=ArmeEquipee, inventaire=ListeArmes)

2. Tests de la classe

- Créer le joueur Arthur avec 50 points de vie et équipé de l'Épée.

Afficher les caractéristiques du joueur Arthur avec la méthode `toString()`.

Résultat attendu :

(Arthur, vie=50, arme=(Épée, 15), inventaire=[(Poing, 5)])

- Créer une nouvelle arme : une Hache avec 20 de dégâts.

Ajouter la Hache et l'Arc dans l'inventaire d'Arthur.

Afficher l'inventaire avec la méthode `afficherInventaire()`.

Résultat attendu :

Inventaire de Arthur :

(Poing, 5)

(Arc, 12)

(Hache, 20)

- Changer l'arme d'Arthur en équipant la Hache.

Résultat attendu :

Arthur a équipé (Hache, 20)

- Changer l'arme d'Arthur avec une arme inexistante (ex : Lance)

Résultat attendu :

Arme non trouvée dans l'inventaire

- Créer un joueur Lancelot avec 50 points de vie et sans arme équipée.
- Afficher les caractéristiques de ce joueur avec la méthode toString().

Résultat attendu :

(Lancelot, vie=50, arme=(Poing, 5), inventaire=[])

- Équiper le joueur Lancelot avec l'arme Arc.
- Afficher l'arme équipé de Lancelot dans un string.

Résultat attendu :

Lancelot est équipé de l'arme : (Arc, 12)

- Afficher l'inventaire mis à jour de Lancelot avec la méthode afficherInventaire().

Résultat attendu :

Inventaire de Lancelot :

(Poing, 5)

III. La classe Combat

1. Implémentation de la classe

Créer une classe Combat avec :

- Deux propriétés :

- **joueur1** : le premier joueur (Joueur – non modifiable).
- **joueur2** : le second joueur (Joueur – non modifiable).

Ces deux propriétés doivent être accessible uniquement dans la classe Combat.

- Trois méthodes :

- **jouerTour(attaquant: Joueur, defenseur: Joueur)** une méthode accessible uniquement dans la classe Combat.
Cette méthode permet à un joueur attaquant d'appliquer les dégâts de son arme sur la vie d'un joueur défenseur. Les messages suivants s'affichent :

***NomAttaquant* attaque *NomDefenseur* (degat dégâts)**

***NomDefenseur* a VieDefenseur PV**

- **afficherVainqueur()** une méthode accessible uniquement dans la classe Combat.
Cette méthode permet d'afficher **Fin du combat !** puis le nom du joueur gagnant ***NomGagnant* a gagné !**
OU Match nul !

- **lancer()** une méthode accessible partout.

Cette méthode permet qu'à chaque tour, le joueur 1 attaque en premier, puis le joueur 2 riposte. Cette alternance continue jusqu'à ce qu'un joueur ait 0 PV.

Cette méthode affiche au début du combat :

Début du combat entre *NomJoueur1* et *NomJoueur2* !

Le numéro du tour est affiché au début de chaque tour.

--- Tour XX ---

Dans cette méthode vous devez appeler les méthodes internes à la classe : **jouerTour(attaquant: Joueur, défenseur: Joueur)** et **afficherVainqueur()**

2. Tests de la classe

- Lancer un combat entre Arthur et Lancelot.

Résultat attendu :

Début du combat entre Arthur et Lancelot !

--- Tour 1 ---

Arthur attaque Lancelot (20 dégâts)

Lancelot a 30 PV

Lancelot attaque Arthur (12 dégâts)

Arthur a 38 PV

--- Tour 2 ---

Arthur attaque Lancelot (20 dégâts)

Lancelot a 10 PV

Lancelot attaque Arthur (12 dégâts)

Arthur a 26 PV

--- Tour 3 ---

Arthur attaque Lancelot (20 dégâts)

Lancelot a 0 PV

Fin du combat !

Arthur a gagné !

IV. Pour aller plus loin (partie non obligatoire)

Il existe certaines incohérences ou bug possible dans ce jeu.

Proposer une solution pour résoudre les problème suivants :

- Incohérence sur l'inventaire

Les deux constructeurs gèrent l'inventaire de manière contradictoire. L'arme équipée n'est pas forcément dans l'inventaire mais cela n'est pas cohérent dans le contexte du jeu.

- Il est possible de créer un objet Arme avec des dégâts négatifs.
- Il est possible de créer un objet Joueur avec une vie négative.
- Le combat n'est très répétitif. Ajouter un coup critique aléatoire dans le combat ou une autre fonctionnalité.
- Il est possible d'avoir des doublons dans l'inventaire.